

DeadlineAI Documentation

DeadlineAI is a proactive, high-fidelity, skeuomorphic, and retro-themed intelligent pocket workspace organizer and task/calendar planner. It is designed to act as an active **Chief-of-Staff** for a user's day, engineered to mitigate procrastination, prevent burnout, correct task underestimation risks, and dynamically structure micro-session schedules.

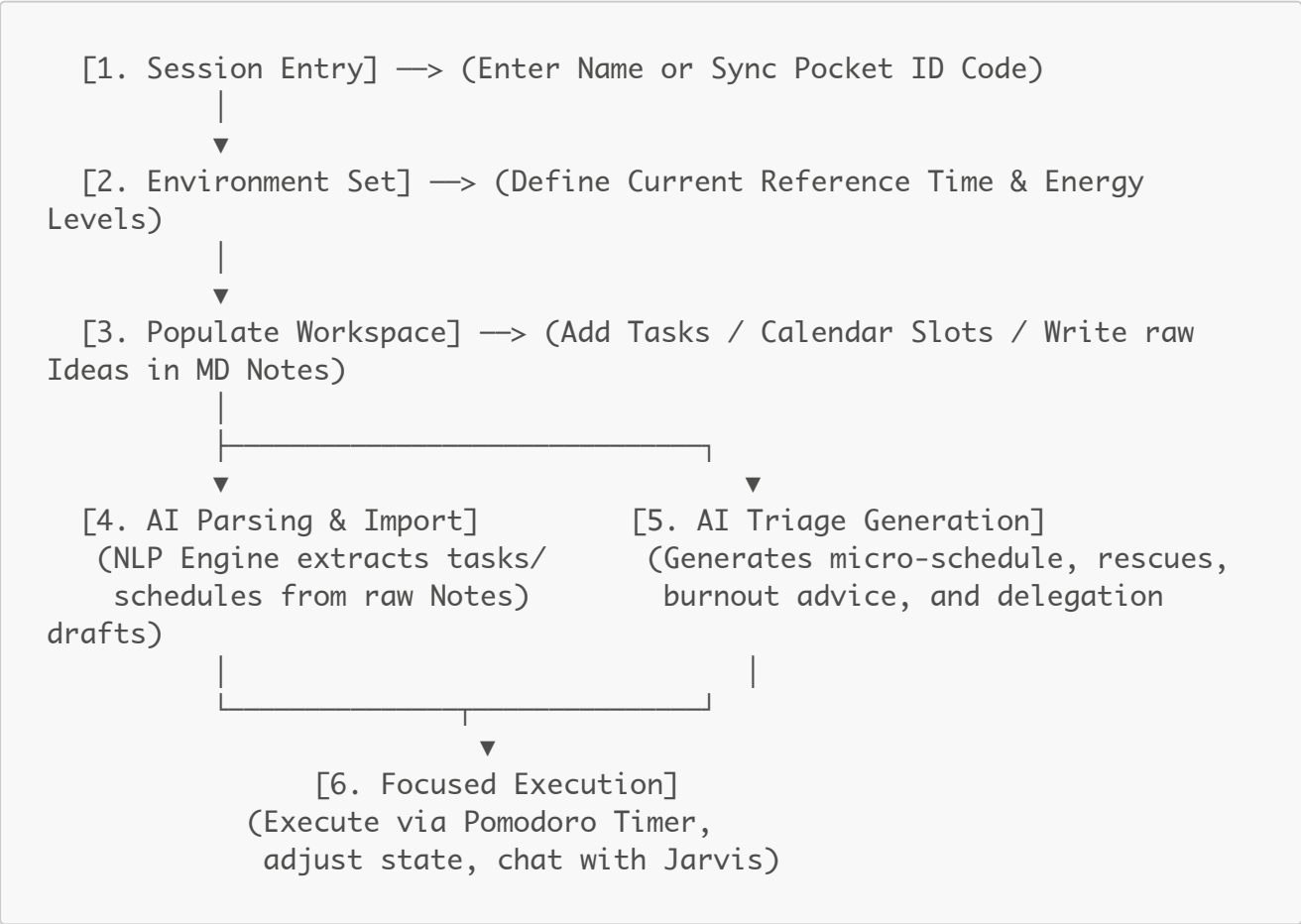
1. Project Overview

At its core, **DeadlineAI** goes beyond passive reminder tools. Traditional planners rely on the user to accurately estimate times, manage focus buffers, and manually sort schedules. In contrast, DeadlineAI uses the **Google Gemini API** to analyze tasks, calendars, reference wall-clock times, and psychological states (such as energy levels and task avoidance history) to output hyper-personalized daily schedules, crisis interventions, and professional delegation emails.

The application features a tactile, responsive **skeuomorphic layout** resembling a classic digital organizer (complete with retro LCD pixel grids, clicking beep sounds, and realistic ruled notebook surfaces) that renders planning physical, structured, and engaging.

2. User Workflow

The typical user journey is structured as follows:



1. Session Entry / Synchronization:

- The user opens the application and enters their name to start a new offline or online workspace session.
- Alternatively, they can input a **6-character alpha-numeric Sync Code** (Pocket ID) to retrieve their exact persisted workspace from the cloud, enabling instant multi-device collaboration or cross-session continuity.

2. Context Setup:

- The user registers their current local time (used as the wall-clock reference for hourly planning) and their active **Energy Level** (**low**, **medium**, or **high**).

3. Workspace Population:

- **Task Management:** The user inputs current deliverables, detailing priority, category, and an estimated duration in hours.
- **Calendar Today:** The user populates structured daily commitments (meetings, gym sessions, stand-ups).
- **Markdown Notes:** The user writes down messy brainstorming, raw notes, or spoken transcripts on the notebook paper pad.

4. Natural Language Processing & Import:

- Upon saving a note, the backend **NLP Engine** analyzes the text, predicts execution risks, extracts structured calendar events and tasks, and opens an **Agenda Extraction Portal** displaying extracted items. The user reviews, updates, and checks or unchecks items to register them directly into the planner.

5. AI Triage Generation:

- The user clicks **AI TRIAGE NOW**. The system analyzes all current tasks, due times, energy constraints, and burnout streaks to formulate:
 - **Critical Now Priority:** The single most high-impact item requiring immediate focus.
 - **Micro-Session Schedule:** An hourly block-by-block layout incorporating task work, meetings, and tactical recovery buffers.
 - **Crisis Rescue Plan:** Active step-by-step mitigation advice if a major deadline is under immediate risk.
 - **Burnout Mitigation:** Coaching advice and scheduled breaks to protect mental health if a streak of missed tasks is flagged.
 - **Procrastination Interceptor:** Direct psychological advice and easy-start actions (e.g. "open report and write one sentence") if task avoidance behaviors are registered.
 - **Delegation Drafts:** Automatically drafted professional emails to delegate low-priority, high-effort tasks.

6. Interactive Focused Execution:

- The user executes tasks using the tactile **Pomodoro LCD Timer** with audio feedback, checks off items, and chats with the embedded **Jarvis Chat** terminal to adjust plans dynamically.
-

3. How to Use DeadlineAI

- **Pocket ID Synchronization:** Use the top-bar status panel to copy your unique alphanumeric sync code or type in a collaborator's code to synchronize states instantly.
 - **Adding/Editing Tasks:** Use the Task Console on the left notebook sheet to insert new tasks. For each task, select a category (e.g., *Coding, Writing, Design, Admin, Meeting, Personal*), designate priority level, and input estimated effort in hours.
 - **Triggering AI Triage:** Keep your current reference time and energy levels updated on the LCD screen, then press the green **AI TRIAGE NOW** button. The central pane will morph to display the generated schedule, critical priorities, burnout warnings, and communication templates.
 - **Tactile Notes Area:** Double-click the Note Sheet to open the editor. Type or paste your ideas. Upon pressing **SAVE NOTES**, a smart overlay will appear, highlighting calendar events and tasks recognized in the text. Click **COMMIT ITEMS** to populate them directly.
 - **Focused Sprints:** Select a task in the list, then activate the skeuomorphic LCD Timer at the bottom right. Click **START** to play the clicking timers, enabling high-performance focused sessions with realistic buzzer/ticking sound effects.
-

4. Architectural Advantages

1. Mitigates Context-Switching Costs:

- The triage agent calculates cognitive friction. If more than three unrelated task categories overlap in a short period, it inserts mandatory buffer blocks and issues a clear alert warning the user of efficiency drops.

2. Compensates for Cognitive Bias (Underestimation Risk):

- Studies indicate humans underestimate task durations by $1.5\times$ to $2.5\times$ (the Planning Fallacy). The natural language processor dynamically doubles predicted effort values for complex categories (e.g., *Coding, Design*) and flags underestimation risks.

3. Combats avoidance via Micro-Action Triggers:

- If energy is low or task progress is stagnant, the agent intercepts avoidance behaviors (e.g., checking emails instead of coding) and prompts the user with micro-activities to break starting inertia.

4. Multi-Model Fallback Resilience:

- Highly resilient server endpoints run a fallback model chain (*gemini-3.1-flash-lite* -> *gemini-flash-latest* -> *gemini-3.5-flash*) to guarantee instant response delivery, even during peak API traffic or temporary service outages.

5. Real-time Synchronization with Local Offline Fallback:

- Leverages Firestore listeners for real-time multiplayer/cross-device sync. If internet connectivity drops, it gracefully falls back to local client state, preventing loading freeze blocks.
-

5. Technology Stack

- **Frontend Framework:** React 18, Vite (TypeScript environment).
- **Styling:** Tailwind CSS (featuring modern, streamlined `@import "tailwindcss"` syntax and a custom warm paper/slate developer palette).
- **Animations:** Framer Motion (`motion/react`) handling high-fidelity deck transitions, sliding modal overlays, and tactile push buttons.
- **Icons:** Lucide React.
- **Backend Server:** Express (Node.js) proxying API requests and securing secrets.
- **Database:** Cloud Firestore (Google Cloud Platform).
- **Authentication:** Firebase Authentication (configured with standard anonymous/guest state options).
- **AI SDK:** Modern `@google/genai` TypeScript SDK.

6. APIs and Data Storage

APIs & Integration Architecture

1. **Vite Middleware:** Mounted within the custom Express server to bundle, hot-reload, and serve static frontend assets transparently.
2. **Gemini Triage API (`/api/triage`):**

 - **Method:** `POST`
 - **Function:** Consumes current session statistics and generates a highly structured JSON response mapping to the client's `AIReport` interface using Gemini's structured output generation schemas.

3. **Gemini Natural Language Parser (`/api/parse-nlp`):**

 - **Method:** `POST`
 - **Function:** Takes raw unstructured text strings (from notes, speech transcripts, or emails) and parses them into organized arrays of tasks and calendar event blocks.

4. **Firestore Real-time Listener (`onSnapshot`):**

 - **Method:** Client-side listener subscribing to `/users/{pocketId}` documents, triggering instantaneous, seamless local state updates when any device modifies the cloud database.

Storage Environment

Data is persisted inside **Google Cloud Firestore**. It uses a single-collection architecture designed to optimize read/write metrics, lower transaction count overhead, and facilitate atomic state updates.

7. Storage Schema

The data schema is structured hierarchically inside Firestore under the `/users` collection, with the unique 6-character alpha-numeric `pocketId` serving as the document identifier.

1. Document Path: `/users/{pocketId}`

Field Name	Type	Description
------------	------	-------------

Field Name	Type	Description
userId	string	Unique internal user UID generated by Firebase Auth or guest session.
username	string	Display name of the user.
currentTime	string	User's wall-clock reference time (e.g. "10:00 AM").
energyLevel	string	Energy configuration: "low", "medium", or "high".
burnoutStreak	number	Consecutive days of missed tasks.
calendarToday	string[]	Array of planned schedule markers (e.g. ["3:00 PM – Team standup (30 min)"]).
dob	string (optional)	Date of birth.
updatedAt	string (optional)	ISO 8601 string representing the last write timestamp.
tasks	Array<Task>	Array of task items (detailed below).
notes	Array<Note>	Array of rich text notes (detailed below).
aiReport	AIReport	Structured object containing AI triage outcomes (detailed below).

Sub-Structures (Types)

Task Entity

```
interface Task {
  id: string; // Unique task ID
  title: string; // Short name of the deliverable
  due: string; // Relative due representation (e.g.
"today 6pm", "tomorrow")
  progress: number; // Numerical value between 0 and 100
  priority: 'low' | 'medium' | 'high';
  completed?: boolean; // Task completion toggle
  category?: string; // e.g. "Coding", "Writing", "Admin",
"Design", "Personal"
  estimatedEffort?: number; // User's estimated hours
  predictedEffort?: number; // AI adjusted predicted hours
  (combating Planning Fallacy)
  underestimationRisk?: boolean; // True if predicted effort diverges
significantly from estimate
}
```

Note Entity

```
interface Note {
  id: string;           // Unique note ID
  title: string;        // Title of the notepad sheet
  content: string;      // Rich raw markdown text content
  updatedAt: string;    // ISO timestamp of modification
}
```

AIReport Entity

```
interface AIReport {
  criticalNow: {
    action: string;           // High-impact active step
  }
  recommendation
  why: string;               // Objective context explaining
  priority selection
};
suggestedSchedule: Array<{
  time: string;              // Hour slot, e.g. "11:15 AM"
  activity: string;          // Recommended verb action or meeting
}
title
  type: 'task' | 'break' | 'meeting';
  taskId?: string;           // Maps to local Task id if
referencing a workspace task
}>;
riskAlert: {
  active: boolean;           // True if deadline failure risk is
triggered
  title: string;             // Heading alert
  why: string;               // Logical analysis of risk
  rescuePlan?: string[];     // Incremental micro-tasks to rescue
the deadline
};
burnoutRisk: {
  active: boolean;           // True if high danger of fatigue is
identified
  message: string;           // Supportive, coaching suggestions to
recover
};
contextSwitchingWarning?: string; // Flag explaining mental depletion
rates due to frequent context shifts
procrastinationIntervention?: {
  active: boolean;           // True if task avoidance is active
  trigger: string;           // Decoded psychological roadblock
  advice: string;            // Direct advice
  suggestion: string;        // Micro-task trigger step
};
delegationSuggestions?: Array<{
  taskTitle: string;         // Low-urgency/high-effort item to
offload
```

```
        delegateTo: string;           // Proposed recipient (e.g. virtual
assistant or colleague)
        emailDraft: string;          // Formatted email template ready to
copy-paste
    }>;
    oneQuestion: string;             // Jarvis clarifying prioritizing
query
    chatResponse: string;           // Formatted initial greeting/response
from Jarvis
}
```